Worked for 30s

# BMW Snowflake Data Warehouse — Step-by-Step Hands-On

We'll build a working **BMW Sales Data Warehouse** in Snowflake from beginning to end.

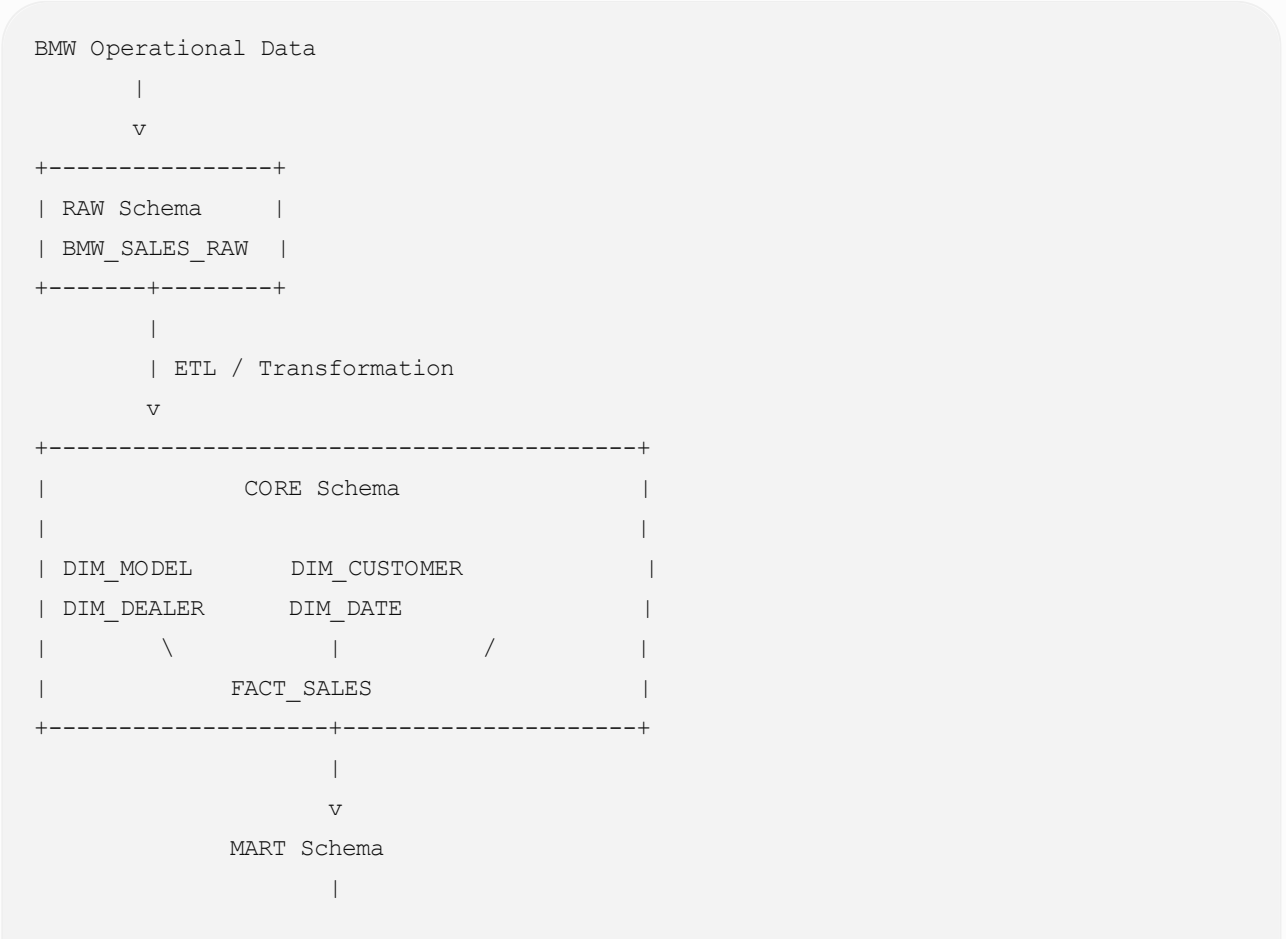
In Snowflake, remember that a **virtual warehouse is compute**, while the actual analytical data warehouse consists of databases, schemas, dimension tables, fact tables, views, and related objects. Snowflake currently supports creating warehouses with options such as size, auto-suspend, and auto-resume; `SYSADMIN` or a higher role has `CREATE WAREHOUSE` by default. Snowflake Docs +1

## 1. Our BMW use case

BMW management wants to answer questions such as:

- Which BMW model generates the most revenue?
- Which cities buy the most vehicles?
- Petrol vs Diesel vs Electric sales?
- Monthly sales trend?
- Which dealers generate the most revenue?
- Which customers purchased which BMW models?

We'll build:



```
                    v
        BI / Dashboard / SQL
```

---

## STEP 2 — Open Snowflake

Login to your Snowflake trial/account and open:

**Projects → Worksheets**

Create a new SQL worksheet.

Run:

```sql
SELECT CURRENT_ACCOUNT(),
       CURRENT_USER(),
       CURRENT_ROLE(),
       CURRENT_WAREHOUSE();
```

---

## STEP 3 — Select SYSADMIN

```sql
USE ROLE SYSADMIN;
```

We'll use `SYSADMIN` for this lab.

---

## STEP 4 — Create ETL virtual warehouse

Create compute for loading and transforming BMW data.

```sql
CREATE OR REPLACE WAREHOUSE BMW_ETL_WH
    WAREHOUSE_SIZE = 'XSMALL'
    AUTO_SUSPEND = 60
    AUTO_RESUME = TRUE
    INITIALLY_SUSPENDED = TRUE;
```

Then:

```sql
USE WAREHOUSE BMW_ETL_WH;
```

Snowflake can automatically resume the warehouse when work arrives and suspend it after the configured idle period. This is useful for controlling unnecessary compute consumption. Snowflake D…

Check it:

```sql
SHOW WAREHOUSES;
```

You should see:

```
BMW_ETL_WH
```

---

## STEP 5 — Create separate BI warehouse

For reporting, create another virtual warehouse.

```sql
CREATE OR REPLACE WAREHOUSE BMW_BI_WH
    WAREHOUSE_SIZE = 'XSMALL'
    AUTO_SUSPEND = 60
    AUTO_RESUME = TRUE
    INITIALLY_SUSPENDED = TRUE;
```

Our architecture now becomes:

```
          | Storage Layer  |
          | BMW_DW         |
          +----------------+
```

The advantage is that an ETL workload doesn't have to compete directly with reporting workloads for the same warehouse compute.

---

## STEP 6 — Create BMW database

```
SQL

USE WAREHOUSE BMW_ETL_WH;

CREATE OR REPLACE DATABASE BMW_DW;
```

Select it:

```
SQL

USE DATABASE BMW_DW;
```

Check:

```
SQL

SHOW DATABASES;
```

---

## STEP 7 — Create warehouse schemas

We'll use three layers:

```
BMW_DW
 |
 +-- RAW
 |
 +-- CORE
 |
 +-- MART
```

Create them:

```
SQL
```

```
CREATE OR REPLACE SCHEMA BMW_DW.RAW;

CREATE OR REPLACE SCHEMA BMW_DW.CORE;

CREATE OR REPLACE SCHEMA BMW_DW.MART;
```

Purpose:

| Schema | Purpose |
|---|---|
| `RAW` | Original source data |
| `CORE` | Dimension and fact tables |
| `MART` | Reporting views / business datasets |

---

## STEP 8 — Create BMW raw sales table

SQL

```
CREATE OR REPLACE TABLE BMW_DW.RAW.BMW_SALES_RAW
(
    SALE_ID          NUMBER,
    CUSTOMER_NAME    VARCHAR(100),
    MODEL            VARCHAR(50),
    FUEL_TYPE        VARCHAR(20),
    CITY             VARCHAR(50),

    DEALER_ID        VARCHAR(20),
    DEALER_NAME      VARCHAR(100),
    DEALER_CITY      VARCHAR(50),

    SALE_DATE        DATE,
    PRICE            NUMBER(12,2),
    QUANTITY         NUMBER
);
```

---

## STEP 9 — Insert BMW sample source data

For the first lab we'll insert data directly. Later you can replace this step with CSV/S3/Azure/GCS loading.

SQL

```
INSERT INTO BMW_DW.RAW.BMW_SALES_RAW
VALUES

(1001,'Rahul Sharma','X5','Petrol','Mumbai',
 'D001','BMW Mumbai Central','Mumbai',
 '2024-01-15',8500000,1),

(1002,'Priya Patel','iX','Electric','Delhi',
 'D002','BMW Delhi Motors','Delhi',
 '2024-02-20',9500000,1),

(1003,'Amit Kumar','3 Series','Diesel','Bangalore',
 'D003','BMW Bangalore','Bangalore',
 '2023-11-05',5200000,1),

(1004,'Sneha Reddy','i4','Electric','Hyderabad',
 'D004','BMW Hyderabad','Hyderabad',
 '2024-03-12',7200000,1),

(1005,'Vikram Singh','X3','Petrol','Chennai',
 'D005','BMW Chennai','Chennai',
 '2024-01-28',6800000,1),

(1006,'Ananya Gupta','iX','Electric','Pune',
 'D006','BMW Pune','Pune',
 '2024-04-02',9700000,1),

(1007,'Rohan Mehta','5 Series','Diesel','Mumbai',
 'D001','BMW Mumbai Central','Mumbai',
 '2023-12-18',6100000,1),

(1008,'Kavita Joshi','X1','Petrol','Delhi',
 'D002','BMW Delhi Motors','Delhi',
 '2024-02-05',4800000,1);
```

Verify:

SQL

```
SELECT *
FROM BMW_DW.RAW.BMW_SALES_RAW;
```

Expected:

```
8 rows
```

Check:

SQL

```
SELECT COUNT(*)
FROM BMW_DW.RAW.BMW_SALES_RAW;
```

---

# STEP 10 — Design the star schema

Our analytical warehouse will look like this:

```
                    DIM_DATE
                       |
                       |
DIM_CUSTOMER ---- FACT_SALES ---- DIM_MODEL
                       |
                       |
                   DIM_DEALER
```

### Dimension tables

Dimensions contain descriptive information.

```
DIM_MODEL
DIM_CUSTOMER
DIM_DEALER
DIM_DATE
```

### Fact table

Fact tables contain measurable business events.

```
FACT_SALES
```

Measures will include:

```
PRICE
QUANTITY
REVENUE
```

---

# STEP 11 — Create DIM_MODEL

```
    SQL

CREATE OR REPLACE TABLE BMW_DW.CORE.DIM_MODEL
(
    MODEL_KEY NUMBER AUTOINCREMENT,
```

```
    MODEL     VARCHAR(50),
    FUEL_TYPE VARCHAR(20),

    CONSTRAINT PK_MODEL PRIMARY KEY (MODEL_KEY)
);
```

Populate it:

SQL

```
INSERT INTO BMW_DW.CORE.DIM_MODEL
(
    MODEL,
    FUEL_TYPE
)
SELECT DISTINCT
       MODEL,
       FUEL_TYPE
FROM BMW_DW.RAW.BMW_SALES_RAW;
```

Check:

SQL

```
SELECT *
FROM BMW_DW.CORE.DIM_MODEL
ORDER BY MODEL_KEY;
```

Example:

```
MODEL_KEY   MODEL       FUEL_TYPE
1           X5          Petrol
2           iX          Electric
3           3 Series    Diesel
...
```

---

# STEP 12 — Create DIM_CUSTOMER

SQL

```
CREATE OR REPLACE TABLE BMW_DW.CORE.DIM_CUSTOMER
(
    CUSTOMER_KEY   NUMBER AUTOINCREMENT,
    CUSTOMER_NAME  VARCHAR(100),
    CITY           VARCHAR(50),
```

```
    CONSTRAINT PK_CUSTOMER PRIMARY KEY (CUSTOMER_KEY)
```

Load it:

```
SQL

INSERT INTO BMW_DW.CORE.DIM_CUSTOMER
(
    CUSTOMER_NAME,
    CITY
)
SELECT DISTINCT
       CUSTOMER_NAME,
       CITY
FROM BMW_DW.RAW.BMW_SALES_RAW;
```

Check:

```
SQL

SELECT *
FROM BMW_DW.CORE.DIM_CUSTOMER;
```

---

# STEP 13 — Create DIM_DEALER

```
SQL

CREATE OR REPLACE TABLE BMW_DW.CORE.DIM_DEALER
(
    DEALER_KEY   NUMBER AUTOINCREMENT,
    DEALER_ID    VARCHAR(20),
    DEALER_NAME  VARCHAR(100),
    DEALER_CITY  VARCHAR(50),

    CONSTRAINT PK_DEALER PRIMARY KEY (DEALER_KEY)
);
```

Load:

```
SQL

INSERT INTO BMW_DW.CORE.DIM_DEALER
(
    DEALER_ID,
    DEALER_NAME,
    DEALER_CITY
)
```

```sql
SELECT DISTINCT
       DEALER_ID,
       DEALER_NAME,
       DEALER_CITY
FROM BMW_DW.RAW.BMW_SALES_RAW;
```

Check:

```sql
SELECT *
FROM BMW_DW.CORE.DIM_DEALER;
```

---

# STEP 14 — Create DIM_DATE

Date dimension:

```sql
CREATE OR REPLACE TABLE BMW_DW.CORE.DIM_DATE
(
    DATE_KEY     NUMBER,
    FULL_DATE    DATE,
    DAY_NUMBER   NUMBER,
    MONTH_NUMBER NUMBER,
    MONTH_NAME   VARCHAR(20),
    QUARTER      NUMBER,
    YEAR         NUMBER,

    CONSTRAINT PK_DATE PRIMARY KEY (DATE_KEY)
);
```

Populate from source sales dates:

```sql
INSERT INTO BMW_DW.CORE.DIM_DATE

SELECT DISTINCT

       TO_NUMBER(
           TO_CHAR(SALE_DATE,'YYYYMMDD')
       ) AS DATE_KEY,

       SALE_DATE,

       DAY(SALE_DATE),
```

```
        MONTH(SALE_DATE),

        MONTHNAME(SALE_DATE),

        QUARTER(SALE_DATE),

        YEAR(SALE_DATE)

FROM BMW_DW.RAW.BMW_SALES_RAW;
```

Check:

```
SQL

SELECT *
FROM BMW_DW.CORE.DIM_DATE
ORDER BY FULL_DATE;
```

Example:

```
DATE_KEY    FULL_DATE    MONTH_NAME   YEAR

20231105    2023-11-05   Nov          2023
20231218    2023-12-18   Dec          2023
20240115    2024-01-15   Jan          2024
```

---

# STEP 15 — Create FACT_SALES

Now create the central warehouse fact table.

```
SQL

CREATE OR REPLACE TABLE BMW_DW.CORE.FACT_SALES
(
    SALE_ID       NUMBER,

    DATE_KEY       NUMBER,
    MODEL_KEY      NUMBER,
    CUSTOMER_KEY   NUMBER,
    DEALER_KEY     NUMBER,

    QUANTITY       NUMBER,
    PRICE          NUMBER(12,2),
    REVENUE        NUMBER(14,2),

    CONSTRAINT PK_SALES PRIMARY KEY (SALE_ID)
);
```

Notice that we don't repeat:

```
Customer name
Model
Fuel type
Dealer name
Month name
```

inside the fact table.

Instead it contains keys.

---

# STEP 16 — ETL raw data into FACT_SALES

This is the important warehouse transformation.

SQL

```sql
INSERT INTO BMW_DW.CORE.FACT_SALES
(
    SALE_ID,
    DATE_KEY,
    MODEL_KEY,
    CUSTOMER_KEY,
    DEALER_KEY,
    QUANTITY,
    PRICE,
    REVENUE
)

SELECT

    R.SALE_ID,

    D.DATE_KEY,

    M.MODEL_KEY,

    C.CUSTOMER_KEY,

    DL.DEALER_KEY,

    R.QUANTITY,

    R.PRICE,

    R.PRICE * R.QUANTITY AS REVENUE
```

```
FROM BMW_DW.RAW.BMW_SALES_RAW R

JOIN BMW_DW.CORE.DIM_MODEL M
  ON R.MODEL = M.MODEL
 AND R.FUEL_TYPE = M.FUEL_TYPE

JOIN BMW_DW.CORE.DIM_CUSTOMER C
  ON R.CUSTOMER_NAME = C.CUSTOMER_NAME
 AND R.CITY = C.CITY

JOIN BMW_DW.CORE.DIM_DEALER DL
  ON R.DEALER_ID = DL.DEALER_ID

JOIN BMW_DW.CORE.DIM_DATE D
  ON R.SALE_DATE = D.FULL_DATE;
```

---

## STEP 17 — Verify the fact table

SQL

```
SELECT *
FROM BMW_DW.CORE.FACT_SALES;
```

Should return:

```
8 rows
```

Run:

SQL

```
SELECT COUNT(*)
FROM BMW_DW.CORE.FACT_SALES;
```

---

## STEP 18 — Validate total BMW revenue

SQL

```
SELECT
    SUM(REVENUE) AS TOTAL_REVENUE
FROM BMW_DW.CORE.FACT_SALES;
```

With our sample data, expected total:

```
57,800,000
```

or:

```
₹5.78 crore
```

---

## STEP 19 — Create the reporting MART

Instead of BI tools repeatedly joining all the tables, create a business-friendly view.

SQL

```sql
CREATE OR REPLACE VIEW BMW_DW.MART.VW_BMW_SALES_ANALYTICS
AS

SELECT

    F.SALE_ID,

    D.FULL_DATE,
    D.MONTH_NAME,
    D.QUARTER,
    D.YEAR,

    M.MODEL,
    M.FUEL_TYPE,

    C.CUSTOMER_NAME,
    C.CITY AS CUSTOMER_CITY,

    DL.DEALER_ID,
    DL.DEALER_NAME,
    DL.DEALER_CITY,

    F.QUANTITY,
    F.PRICE,
    F.REVENUE

FROM BMW_DW.CORE.FACT_SALES F

JOIN BMW_DW.CORE.DIM_DATE D
ON F.DATE_KEY = D.DATE_KEY

JOIN BMW_DW.CORE.DIM_MODEL M
ON F.MODEL_KEY = M.MODEL_KEY

JOIN BMW_DW.CORE.DIM_CUSTOMER C
```

```
ON F.CUSTOMER_KEY = C.CUSTOMER_KEY

JOIN BMW_DW.CORE.DIM_DEALER DL
ON F.DEALER_KEY = DL.DEALER_KEY;
```

---

## STEP 20 — Switch to the BI warehouse

ETL is complete.

Now analytics users use:

```
SQL

USE WAREHOUSE BMW_BI_WH;
```

Then:

```
SQL

USE DATABASE BMW_DW;
```

This demonstrates an important Snowflake architecture concept:

```
BMW_ETL_WH
     |
     | loads/transforms
     v
   BMW_DW
     ^
     | queries
     |
BMW_BI_WH
```

Both compute warehouses operate against the same centralized Snowflake data.

---

## STEP 21 — Query total sales

```
SQL

SELECT
    COUNT(*) AS TOTAL_VEHICLES,
    SUM(REVENUE) AS TOTAL_REVENUE
FROM BMW_DW.MART.VW_BMW_SALES_ANALYTICS;
```

---

## STEP 22 — BMW revenue by model

```sql
SELECT
    MODEL,
    SUM(QUANTITY) AS VEHICLES_SOLD,
    SUM(REVENUE) AS REVENUE
FROM BMW_DW.MART.VW_BMW_SALES_ANALYTICS
GROUP BY MODEL
ORDER BY REVENUE DESC;
```

You should find `iX` at or near the top with this sample because it appears twice.

---

## STEP 23 — Petrol vs Diesel vs Electric

```sql
SELECT
    FUEL_TYPE,
    COUNT(*) AS SALES,
    SUM(REVENUE) AS REVENUE
FROM BMW_DW.MART.VW_BMW_SALES_ANALYTICS
GROUP BY FUEL_TYPE
ORDER BY REVENUE DESC;
```

This is the sort of query a BMW management dashboard could use.

---

## STEP 24 — Electric vehicle revenue

```sql
```

```sql
SELECT
    MODEL,
    SUM(REVENUE) AS EV_REVENUE
FROM BMW_DW.MART.VW_BMW_SALES_ANALYTICS
WHERE FUEL_TYPE = 'Electric'
GROUP BY MODEL
ORDER BY EV_REVENUE DESC;
```

---

## STEP 25 — Revenue by city

SQL

```sql
SELECT
    CUSTOMER_CITY,
    COUNT(*) AS TOTAL_SALES,
    SUM(REVENUE) AS TOTAL_REVENUE
FROM BMW_DW.MART.VW_BMW_SALES_ANALYTICS
GROUP BY CUSTOMER_CITY
ORDER BY TOTAL_REVENUE DESC;
```

---

## STEP 26 — Dealer performance

SQL

```sql
SELECT
    DEALER_NAME,
    DEALER_CITY,
    COUNT(*) AS VEHICLES_SOLD,
    SUM(REVENUE) AS REVENUE
```

```
FROM BMW_DW.MART.VW_BMW_SALES_ANALYTICS

GROUP BY
    DEALER_NAME,
    DEALER_CITY

ORDER BY REVENUE DESC;
```

This gives BMW a dealer-performance report.

---

## STEP 27 — Monthly BMW sales trend

SQL

```
SELECT

    YEAR,

    MONTH(FULL_DATE) AS MONTH_NUMBER,

    MONTH_NAME,

    COUNT(*) AS VEHICLES_SOLD,

    SUM(REVENUE) AS REVENUE

FROM BMW_DW.MART.VW_BMW_SALES_ANALYTICS

GROUP BY
    YEAR,
    MONTH(FULL_DATE),
    MONTH_NAME

ORDER BY
    YEAR,
    MONTH_NUMBER;
```

---

## STEP 28 — Customer purchase report

SQL

```
SELECT

    CUSTOMER_NAME,
```

```
    MODEL,

    FUEL_TYPE,

    FULL_DATE,

    PRICE

FROM BMW_DW.MART.VW_BMW_SALES_ANALYTICS

ORDER BY FULL_DATE;
```

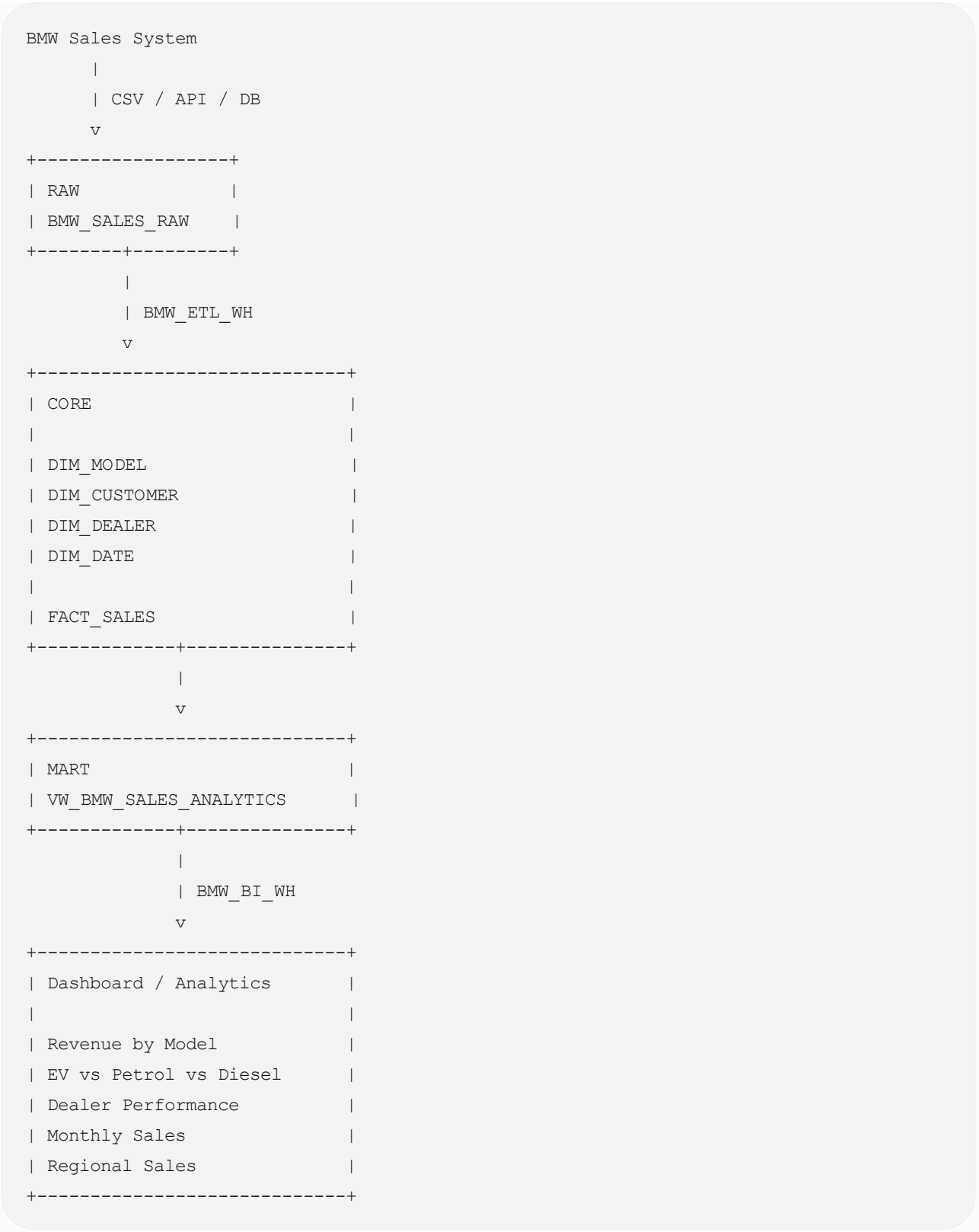
---

# Final BMW warehouse

You have now created:

```
SNOWFLAKE ACCOUNT
│
├── COMPUTE
│
│   ├── BMW_ETL_WH
│   │       └── Data loading / transformation
│   │
│   └── BMW_BI_WH
│           └── Reporting / analytics
│
└── BMW_DW
     │
     ├── RAW
     │    │
     │    └── BMW_SALES_RAW
     │
     ├── CORE
     │    │
     │    ├── DIM_MODEL
     │    ├── DIM_CUSTOMER
     │    ├── DIM_DEALER
     │    ├── DIM_DATE
     │    │
     │    └── FACT_SALES
     │
     └── MART
          │
          └── VW_BMW_SALES_ANALYTICS
```

## Complete data flow




## What you have learned

This single lab covers the main data-warehouse concepts:

**Source → Raw → ETL → Dimensions → Fact → Star Schema → Data Mart → BI Queries**, while also demonstrating Snowflake's separation of storage from independent virtual warehouses for ETL and BI workloads. Snowflake's current warehouse configuration supports the `WAREHOUSE_SIZE`, `AUTO_SUSPEND`, `AUTO_RESUME`, and related properties used in the lab. Snowflake D…

The natural next step is to turn this into a more realistic **BMW enterprise warehouse** with four source domains:

```
Sales
Vehicle Telemetry
Dealer
Maintenance
        ↓
Snowflake RAW
        ↓
STAGING
        ↓
DIMENSIONS + FACT TABLES
        ↓
Sales Mart
Vehicle Mart
Maintenance Mart
Dealer Mart
        ↓
BI / ML
```

That version would include `FACT_SALES`, `FACT_TELEMETRY`, `FACT_MAINTENANCE`, `DIM_VEHICLE`, `DIM_CUSTOMER`, `DIM_DEALER`, and `DIM_DATE` and gives you a complete BMW Snowflake project suitable for training or demonstration.